# STM32F103C8T6 移植 uC/OS-III 并采集 AHT20 温湿度

## 1. 实验目标与总体方案

本实验包含两个部分。第一部分学习 uC/OS-III 的移植方法，完成三个并发任务：Task1 每 1 秒控制板载 LED，Task2 每 3 秒控制外接 LED并输出运行信息，Task3 每 2 秒执行一次传感器采集。第二部分学习 $I^2C$ 总线和 AHT20 数据格式，将采集结果通过 USART1 发送到 Windows 10 串口助手。

最终工程使用 STM32F103C8T6、HAL 库、Keil5 和 uC/OS-III。软件 $I^2C$ 使用 PB6/PB7，串口使用 USART1 的 PA9/PA10。

## 2. 实验环境与硬件连接

### 2.1 硬件和软件环境

| 项目 | 配置 |
| --- | --- |
| MCU | STM32F103C8T6 |
| 开发方式 | STM32CubeMX + HAL + Keil5 |
| 实时操作系统 | uC/OS-III |
| 传感器 | AHT20 |
| 下载器 | ST-Link |
| 串口模块 | USB-TTL |
| 串口参数 | 115200 baud，8 数据位，无校验，1 停止位 |

### 2.2 AHT20 与 $I^2C$ 接线

根据传感器资料，四个引脚定义为：1-VDD、2-SDA、3-GND、4-SCL。

PB6/PB7 使用开漏方式工作，数据线和时钟线必须通过上拉回到高电平。实际接线和传感器引脚图见下方占位图。

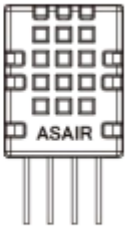
| 引脚 | 名称 | 释义 | |
|---|---|---|---|
| 1 | VDD | 接电源(2.2-5.5V) |  |
| 2 | SDA | 串行数据，双向 | |
| 3 | GND | 电源地 | |
| 4 | SCL | 串行时钟，双向 | |



表4. DHT20引脚分布（俯视图）

### 2.3 USART1 与 LED 接线

| STM32 | USB-TTL |
|---|---|
| PA9 / USART1_TX | USB-TTL RX |
| PA10 / USART1_RX | USB-TTL TX |
| GND | GND |

## 3. CubeMX 工程配置

### 3.1 GPIO 配置

对 `PC13` 选择：

```
GPIO_Output
```

3. 左侧展开：

```
System Core → GPIO
```

4. 找到 `PC13`，设置：

```
GPIO output level: High
GPIO mode: Output Push Pull
Maximum output speed: Low
User Label: LED
```

很多 STM32F103C8T6 开发板的板载 LED 是低电平点亮，因此程序中可能需要：

```
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
```

由于我们先学习软件 I2C，所以不选择 `I2C1` 外设。

在芯片图中：

1. 点击 `PB6`
2. 选择：

```
GPIO_Output
```

3. 点击 `PB7`
4. 选择：

```
GPIO_Output
```

然后进入：

```
System Core → GPIO
```

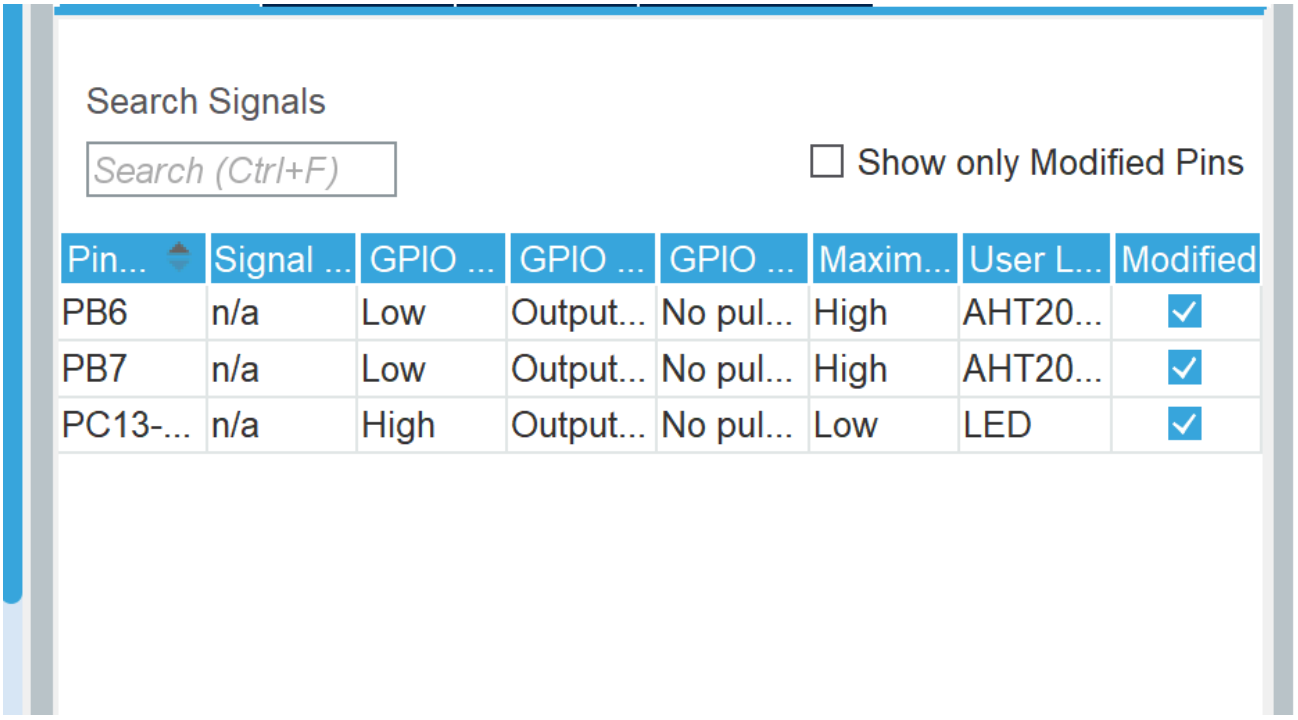
将两个引脚设置为：

```
PB6：
GPIO mode：Output Open Drain
Maximum output speed：High
User Label：AHT20_SCL

PB7：
GPIO mode：Output Open Drain
Maximum output speed：High
User Label：AHT20_SDA
```

AHT20 接线：

```
AHT20 VCC → 3.3V
AHT20 GND → GND
AHT20 SCL → PB6
AHT20 SDA → PB7
```




## 3.2 USART1 配置

1. 左侧点击：

```
Connectivity → USART1
```

2. 选择：

```
Mode: Asynchronous
```

这样通常会自动分配：

```
PA9  → USART1_TX
PA10 → USART1_RX
```

3. 点击左侧：

```
System Core → USART1
```

设置参数：

```
Baud Rate: 115200
Word Length: 8 Bits
Parity: None
Stop Bits: 1
Mode: TX_RX
Hardware Flow Control: None
Over Sampling: 16
```

串口接线为：

```
STM32 PA9  → USB-TTL RX
STM32 PA10 → USB-TTL TX
STM32 GND  → USB-TTL GND
```

USB-TTL 必须使用 `3.3V` 电平，不能把 `5V TTL` 直接接到 STM32 的串口引脚。

## 3.3 系统时钟配置

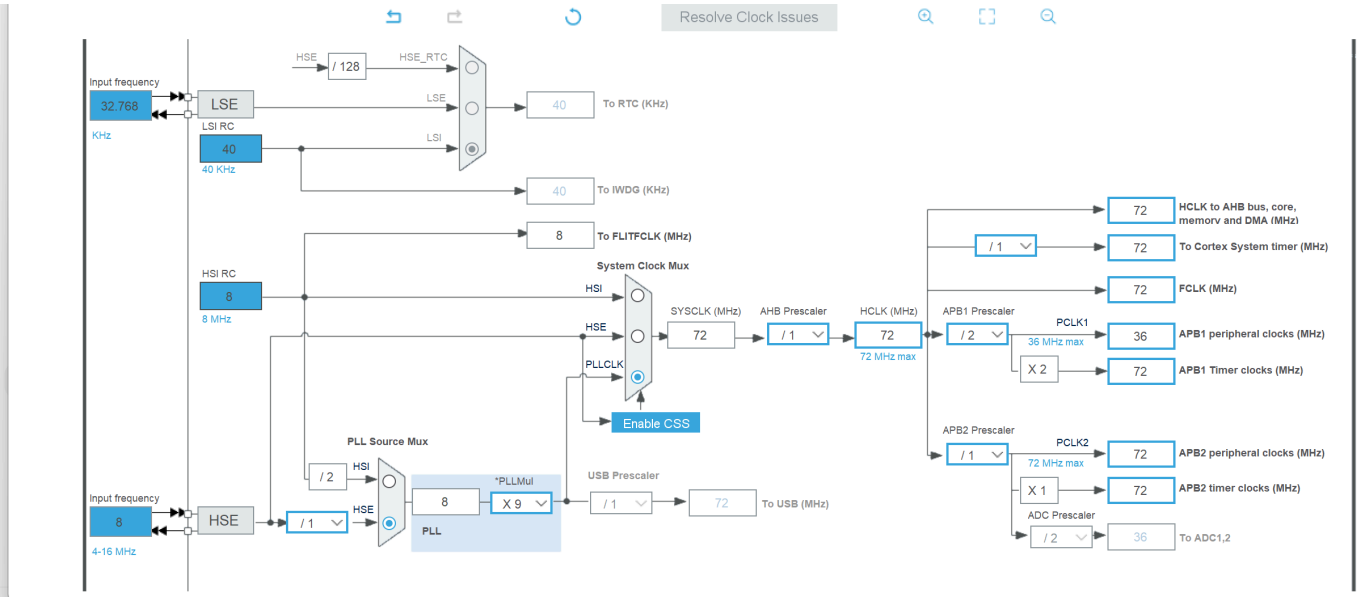
点击顶部：

```
Clock Configuration
```

如果你的开发板有 `8 MHz` 外部晶振：

1. 将系统时钟配置为 `72 MHz`
2. 确认：

```
HCLK  = 72 MHz
PCLK1 = 36 MHz
PCLK2 = 72 MHz
```




## 3.4 Keil 工程生成

# 点击顶部：

```
Project Manager
```

设置：

```
Project Name：ucos_aht20
Project Location：选择一个容易找到的文件夹
Toolchain / IDE：MDK-ARM
生成独立的 .c/.h 文件：保持默认
```

建议工程目录不要放在中文路径中，例如：

```
D:\STM32\ucos_aht20
```

完成后，先不要点击生成代码，发我一张包含以下内容的截图：

- `PC13` 已设置为 GPIO 输出
- `PA9/PA10` 显示 USART1
- `PB6/PB7` 已设置为 GPIO 输出
- 左侧 GPIO 配置页面

确认无误后，再点击右上角：

```
GENERATE CODE
```

生成代码后，我们先只编译并下载一个 LED 闪烁程序，证明 CubeMX、Keil、ST-Link 和开发板都正常，再加入串口和 AHT20。

# 4. uC/OS-III 移植

## 4.1 准备工程目录

在现有工程目录下建立文件夹，其中：

- `Source`：存放 uC/OS-III 内核源码
- `Port`：存放 Cortex-M3 移植文件
- `Cfg`：存放应用配置文件和钩子文件




## 4.2 添加 uC/OS-III 内核源码

从 uC/OS-III 源码目录将所有 `.c` 文件复制到工程中的：uCOS-III\Source里

需要包含的核心文件至少有：

```
os_core.c
os_task.c
os_time.c
os_sem.c
os_mutex.c
os_flag.c
os_q.c
os_msg.c
os_mem.c
os_stat.c
os_tmr.c
os_tick.c
os_prio.c
os_dbg.c
os_cfg_app.c
__dbg_uCOS-III.c
```

## 4.3 添加 Cortex-M3 移植文件

复制 `os_cpu_c.c`，从 ARM 汇编目录复制 `os_cpu_a.asm` 和 `os_cpu.h`：将这三个文件放入 uCOS-III\Port里

最终目录应包含：

```
Port
├── os_cpu_c.c
├── os_cpu_a.asm
└── os_cpu.h
```

## 4.4添加配置模板

从模板目录：Cfg\Template

复制以下文件到工程中的 `Cfg` 目录：

```
os_cfg.h
os_cfg_app.h
os_app_hooks.c
os_app_hooks.h
```

uC/OS-III 移植还依赖配套组件 uC/CPU 和 uC/LIB。

uC/LIB 官方源码

将 uC/CPU 源码放在：

```
D:\STM32\Project\ucos_aht20\uCOS-III\uC-CPU-develop
```

再从官方 uC/LIB 仓库下载 ZIP 并解压到：

```
D:\STM32\Project\ucos_aht20\uCOS-III
```

## 4.5 在 Keil 中添加工程分组

完成文件复制后，在 Keil 左侧工程树中右键工程名 `ucos_aht20`，选择 **Manage Project Items...**。

在弹出的窗口中创建三个分组：

```
uCOS-III Source
uCOS-III Port
uCOS-III Config
```

然后分别添加文件：

- 在 `uCOS-III Source` 中添加 `Source` 目录下的 `.c` 文件。
- 在 `uCOS-III Port` 中添加 `os_cpu_c.c` 和 `os_cpu_a.asm`。
- 在 `uCOS-III Config` 中添加 `os_app_hooks.c`。

如果 Keil 询问是否将文件复制到工程目录，选择 **No**，让工程继续使用当前文件路径。

`os_cfg.h`、`os_cfg_app.h` 和 `os_app_hooks.h` 是头文件，不需要作为源文件加入分组，但必须配置头文件搜索路径。

## 4.6 配置头文件搜索路径

打开：

```
Project → Options for Target → C/C++ → Include Paths
```

添加以下目录：

```
D:\STM32\Project\ucos_aht20\uCOS-III\Port
D:\STM32\Project\ucos_aht20\uCOS-III\Cfg
D:\STM32\Project\ucos_aht20\uCOS-III\Source
```

到这里先不要编译。确认 Keil 工程中已经出现三个新分组，并检查对应文件是否加入正确。

## 4.7 修改中断处理文件

打开 `stm32f1xx_it.c`，在文件顶部加入：

```
#include "os.h"
```

将 `PendSV_Handler()` 修改为：

```
void PendSV_Handler(void)
{
    OS_CPU_PendSVHandler();
}
```

将 `SysTick_Handler()` 修改为：

```
void SysTick_Handler(void)
{
    HAL_IncTick();
    OS_CPU_SysTickHandler();
}
```

`SVC_Handler()` 保持原样，不需要修改。这里由 SysTick 驱动 uC/OS-III 的系统节拍，PendSV 则用于执行任务切换。

## 4.8 添加应用头文件并定义任务控制块和任务栈

打开 `main.c`，在：

```
#include "gpio.h"
```

后面添加：

```
#include "os.h"
#include <stdio.h>
#include <string.h>
```

在 `/* USER CODE BEGIN PV */` 与 `/* USER CODE END PV */` 之间加入：

```
#define APP_TASK_STK_SIZE 256u

static OS_TCB task1_tcb;
static OS_TCB task2_tcb;
static OS_TCB task3_tcb;

static CPU_STK task1_stk[APP_TASK_STK_SIZE];
static CPU_STK task2_stk[APP_TASK_STK_SIZE];
static CPU_STK task3_stk[APP_TASK_STK_SIZE];

static void AppTask1(void *p_arg);
static void AppTask2(void *p_arg);
static void AppTask3(void *p_arg);
```

每个任务都需要一个任务控制块（TCB）和独立的任务栈。这里任务栈大小设为 256 个 `CPU_STK` 单位。

## 4.9编写基础阶段任务函数

uC/OS-III 移植完成后，先编写一组基础任务，用于验证任务创建、系统节拍和任务延时是否正常。这个阶段的 Task1 每秒翻转一次 PC13，Task2 每隔 3 秒发送一次测试文字，Task3 每隔 2 秒

发送一次测试文字。此时 Task2 尚未控制 PA8，Task3 也尚未接入 AHT20。

基础任务运行正常后，再在第 5 章中完善任务功能：Task2 增加 PA8 LED 控制，Task3 改为采集 AHT20 温湿度并通过串口输出。

在 `/* USER CODE BEGIN 0 */` 区域加入三个任务函数：

```c
static void AppTask1(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;

    while (1)
    {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        OSTimeDlyHMSM(0, 0, 1, 0, &err);
    }
}

static void AppTask2(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;

    while (1)
    {
        HAL_UART_Transmit(&huart1,
                          (uint8_t *)"Task2\r\n",
                          7,
                          HAL_MAX_DELAY);

        OSTimeDlyHMSM(0, 0, 3, 0, &err);
    }
}

static void AppTask3(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;

    while (1)
    {
        HAL_UART_Transmit(&huart1,
                          (uint8_t *)"hello uc/OS!\r\n",
                          14,
                          HAL_MAX_DELAY);

        OSTimeDlyHMSM(0, 0, 2, 0, &err);
    }
}
```

在这一阶段，三个任务分别以不同的时间间隔运行，用于确认 uC/OS-III 的任务创建和延时调度正常。此处的 Task2 和 Task3 是基础测试版本，最终任务功能见第 5 章。

## 4.10 编译与基础运行验证

在 `main()` 中，完成 GPIO 和 USART 初始化之后加入 uC/OS-III 初始化、任务创建和调度器启动代码：

```
OS_ERR err;

OSInit(&err);

OSTaskCreate(&task1_tcb,
             "Task1",
             AppTask1,
             0,
             5,
             &task1_stk[0],
             APP_TASK_STK_SIZE / 10,
             APP_TASK_STK_SIZE,
             0,
             0,
             0,
             OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
             &err);

OSTaskCreate(&task2_tcb,
             "Task2",
             AppTask2,
             0,
             6,
             &task2_stk[0],
             APP_TASK_STK_SIZE / 10,
             APP_TASK_STK_SIZE,
             0,
             0,
             0,
             OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
             &err);

OSTaskCreate(&task3_tcb,
             "Task3",
             AppTask3,
             0,
             7,
             &task3_stk[0],
             APP_TASK_STK_SIZE / 10,
             APP_TASK_STK_SIZE,
             0,
             0,
             0,
             OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
             &err);

OSStart(&err);
```

任务优先级分别设置为 5、6、7。完成 `OSStart()` 后，uC/OS-III 开始调度任务，主函数中原有的 HAL 测试循环不再需要。可以保留一个空循环作为兜底：

```
while (1)
{
}
```

### 4.11 编译、下载与运行观察

保存工程并编译，先验证 uC/OS-III 基础任务版本。如果编译出现错误，先根据第一条错误定位问题，再继续处理后续错误。

编译成功后，使用 Keil 的 **Download** 按钮将程序烧录到开发板，并打开串口助手，设置为：

```
115200，8-N-1
```

观察以下现象：

- PC13 LED 是否大约每秒翻转一次；
- 串口是否按任务设定的周期输出信息。

## 5. 三任务设计与验证

本章在此基础上完善任务功能：Task1 继续控制 PC13；Task2 增加 PA8 外接 LED 控制；Task3 则从发送测试文字改为每 2 秒读取一次 AHT20，并通过 USART1 输出温湿度数据。

### 5.1 Task1：PC13 LED 控制

板载 LED 使用 PC13，由 Task1 控制。

在 `/* USER CODE BEGIN 0 */` 区域加入 Task1 任务函数：

```
static void AppTask1(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;

    while (1)
    {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        OSTimeDlyHMSM(0, 0, 1, 0, &err);
    }
}
```

Task1 每隔 1 秒翻转一次 PC13 LED。编译并下载程序后，观察 PC13 LED 是否大约每秒翻转一次。

## 5.2 Task2：PA8 LED 与状态输出

第二个 LED 接在 PA8。PA8 经 220 Ω～1 kΩ 电阻连接 LED 正极，LED 负极接 GND，由 Task2 控制。

Task2：每 3 秒翻转一次 PA8 外接 LED，并通过 USART1 输出运行信息。

在 `/* USER CODE BEGIN 0 */` 区域加入以下任务函数：

```
static void AppTask2(void *p_arg)
{
    OS_ERR err;
    const char msg[] = "Task2 alive\r\n";
    (void)p_arg;

    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);

        HAL_UART_Transmit(&huart1,
                          (uint8_t *)msg,
                          sizeof(msg) - 1u,
                          HAL_MAX_DELAY);

        OSTimeDlyHMSM(0, 0, 3, 0, &err);
    }
}
```

Task2 运行时，首先调用：

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);
```

翻转 PA8 的输出电平，使外接 LED 改变亮灭状态。随后通过 USART1 发送字符串：

```
Task2 alive
```

最后调用：

```
OSTimeDlyHMSM(0, 0, 3, 0, &err);
```

延时 3 秒，延时结束后再次执行上述操作。

因此，Task2 每隔 3 秒改变一次 PA8 的输出电平，并发送一次运行信息。需要注意的是，`HAL_GPIO_TogglePin()` 每 3 秒只改变一次电平，因此 LED 完整经历一次"亮—灭"过程需要约 6 秒。

编译并下载程序后，观察 PA8 外接 LED 是否每隔 3 秒改变一次亮灭状态，同时打开串口助手，设置串口参数为：

```
115200, 8-N-1
```

观察串口是否周期性输出：

```
Task2 alive
```

## 5.3 Task3：传感器采集任务

在 `main.c` 顶部包含 AHT20 驱动头文件：

```
#include "aht20.h"
```

如果任务函数要使用 `snprintf()` 格式化字符串，还要包含：

```
#include <stdio.h>
```

同时确认串口句柄 `huart1` 在当前文件中可用。若 `huart1` 定义在其他模块，应包含对应的 USART 头文件，不能仅凭变量名假定它已经声明。

Task3 启动时先初始化 AHT20，之后每隔 2 秒读取一次温湿度，并通过 USART1 输出采集结果或错误信息。串口发送使用 `UART_SendString()`，该函数和互斥量的定义见 5.4 节。

下面是 Task3 的完整实现：

```c
static void AppTask3(void *p_arg)
{
    OS_ERR err;
    HAL_StatusTypeDef status;
    float temperature = 0.0f;
    float humidity = 0.0f;
    char message[64];

    (void)p_arg;

    status = AHT20_Init();

    if (status != HAL_OK)
    {
        int length = snprintf(message,
                              sizeof(message),
                              "AHT20 init failed, HAL status=%d\r\n",
                              (int)status);

        if (length > 0)
        {
            UART_SendString(message, (uint16_t)length);
        }
    }

    while (1)
    {
        if (status == HAL_OK)
        {
            status = AHT20_Read(&temperature, &humidity);
        }

        if (status == HAL_OK)
        {
            int length = snprintf(message,
                                  sizeof(message),
                                  "AHT20: T=%.2f C, RH=%.2f%%\r\n",
                                  temperature,
                                  humidity);

            if (length > 0)
            {
                UART_SendString(message, (uint16_t)length);
            }
        }
        else
        {
            int length = snprintf(message,
```

```
                                    sizeof(message),
                                    "AHT20 read failed, HAL status=%d\r\n",
                                    (int)status);

            if (length > 0)
            {
                UART_SendString(message, (uint16_t)length);
            }
        }

        OSTimeDlyHMSM(0, 0, 2, 0, &err);
    }
}
```

这段任务代码在初始化失败后会一直报告错误。若想让它每次读取前都尝试重新初始化，可在失败分支中加入：

```
status = AHT20_Init();
```

## 5.4 任务优先级、周期与串口互斥

Task2 和 Task3 共用 USART1。即使每个任务的串口发送都是同步调用，也可能在一个任务发送期间发生任务切换，造成消息交错。解决方法是让发送任务在使用串口前获取互斥量，发送完成后释放互斥量。

在 `main.c` 中定义互斥量：

```
static OS_MUTEX uart_mutex;
```

在 `OSInit()` 之后、创建并启动任务之前创建它：

```
OS_ERR err;

OSInit(&err);

OSMutexCreate(&uart_mutex,
              "UART Mutex",
              &err);
```

然后定义一个统一的串口发送函数：

```c
static void UART_SendString(const char *text, uint16_t length)
{
    OS_ERR err;

    OSMutexPend(&uart_mutex,
                0,
                OS_OPT_PEND_BLOCKING,
                0,
                &err);

    HAL_UART_Transmit(&huart1,
                      (uint8_t *)text,
                      length,
                      HAL_MAX_DELAY);

    OSMutexPost(&uart_mutex,
                OS_OPT_POST_NONE,
                &err);
}
```

这段函数的工作顺序是：

1. `OSMutexPend()`：等待并取得串口互斥量。
2. `HAL_UART_Transmit()`：独占 USART1，发送本条消息。
3. `OSMutexPost()`：发送完成后释放互斥量。

如果等待或释放互斥量时 `err` 不是 `OS_ERR_NONE`，实际工程中应进一步处理错误，而不是假定操作必然成功。

现在将 Task3 中的发送位置替换为：

```c
if (length > 0)
{
    UART_SendString(message, (uint16_t)length);
}
```

Task2 也必须使用同一个 `UART_SendString()`，否则它仍可能绕过互斥保护。例如：

```c
static void AppTask2(void *p_arg)
{
    OS_ERR err;
    static const char message[] = "Task2 alive\r\n";

    (void)p_arg;

    while (1)
    {
        UART_SendString(message, sizeof(message) - 1u);
        OSTimeDlyHMSM(0, 0, 3, 0, &err);
    }
}
```

如果 Task2 仍直接调用 `HAL_UART_Transmit()`，那么只保护 Task3 并不能避免两个任务的串口输出交错。

# 6. 软件 I²C 与 AHT20 数据采集

## 6.1 软件 I²C 和硬件 I²C 的区别

硬件 I²C 使用 STM32 内部专用 I²C 外设，由硬件自动完成时钟、起始停止条件、移位和应答处理，CPU 占用较少，速度和时序更稳定。

软件 I²C 使用普通 GPIO，由程序手动产生 SCL 和 SDA 的高低电平、起始条件、停止条件、字节发送、字节读取和 ACK。它不依赖特定 I²C 外设，移植灵活，但时序由软件延时保证，CPU 占用较高。

本实验采用软件 I²C，PB6 为 SCL，PB7 为 SDA。GPIO 设置为开漏输出，写高表示释放总线，写低表示主动拉低总线。

## 6.2 AHT20 引脚与驱动文件

本工程使用软件 I2C：

```
SCL → PB6
SDA → PB7
```

CubeMX 中需要将 PB6、PB7 配置为开漏输出。开漏模式下，GPIO 输出高电平的含义是“释放线路”，线路实际变高依赖上拉电阻。因此要确认 AHT20 模块已带上拉电阻，或者在 SCL、SDA 上分别外接上拉电阻到 3.3 V。

在 Keil 工程中还要加入驱动源文件，否则即使 `aht20.c` 已经复制到磁盘目录，也不会参与编译：

```
Core/Inc/aht20.h
Core/Src/aht20.c
```

代码中的 I2C 起始和停止信号

I2C 的 START 和 STOP 都通过 SDA 在 SCL 为高时的电平变化表示：

```
/* START：SCL 为高时，SDA 从高变低。 */
I2C_SDA_Low();

/* STOP：SCL 为高时，SDA 从低变高。 */
I2C_SDA_High();
```

这也是软件 I2C 中容易出错的地方。如果 SDA、SCL 的方向或上拉配置不正确，传感器可能不会应答。

## 写入字节与 ACK

发送完 8 位数据后，主机需要释放 SDA，并在第 9 个时钟周期读取从机的应答：

```
I2C_SCL_Low();
I2C_SDA_High();       /* 释放 SDA */
I2C_SCL_High();

uint8_t ack = (I2C_SDA_Read() == GPIO_PIN_RESET);
```

从机将 SDA 拉低表示 ACK。若 SDA 保持高电平，通常表示没有设备应答，可能是地址、接线、供电或上拉存在问题。

## 读取字节时的 ACK 和 NACK

读完一个字节后，主机要回应传感器：

- 还有数据要读：发送 ACK，也就是拉低 SDA。
- 已经读到最后一个字节：发送 NACK，也就是释放 SDA。

因此 `I2C_ReadByte()` 用参数 `ack` 区分这两种情况。最后一个数据字节必须以 NACK 结束本次读取。

## 温湿度换算

传感器返回的湿度和温度不是直接的浮点数，而是 20 位原始值。换算公式为：

```
RH = raw_humidity × 100 / 2²⁰
T  = raw_temperature × 200 / 2²⁰ − 50
```

驱动把结果写入 `temperature` 和 `humidity` 指向的变量。例如：

```
float temperature;
float humidity;

if (AHT20_Read(&temperature, &humidity) == HAL_OK)
{
    /* 读取成功，可使用两个结果 */
}
```

## 6.3 驱动接口和关键实现

头文件负责告诉其他模块驱动提供哪些函数。任务代码只需包含 `aht20.h`，不必知道 I2C 的每个时序细节。

创建 `Core/Inc/aht20.h`：

```
#ifndef AHT20_H
#define AHT20_H

#include "stm32f1xx_hal.h"

HAL_StatusTypeDef AHT20_Init(void);
HAL_StatusTypeDef AHT20_Read(float *temperature, float *humidity);

#endif
```

这里使用 `HAL_StatusTypeDef` 作为返回类型：

- `HAL_OK`：初始化或读取成功。
- 其他 HAL 状态：操作失败。

`AHT20_Read()` 的两个参数是输出指针。调用者传入温度和湿度变量的地址，驱动读取成功后将结果写入这些变量。

## 6.4 实现软件 I²C 和 AHT20 驱动

`aht20.c` 驱动层主要负责两类工作：

1. 用 GPIO 模拟 I2C 的起始、停止、写字节和读字节。
2. 发送 AHT20 命令、取得原始数据并换算成摄氏温度和相对湿度。

下面是参考代码。GPIO 端口和引脚宏按本工程设置为 PB6、PB7；如果你的工程中宏名称不同，应统一替换为实际名称。

```c
#include "aht20.h"

/*
 * 本例使用 PB6 作为 SCL，PB7 作为 SDA。
 * GPIO 写高表示释放开漏线路；写低表示主动拉低。
 */
#define AHT20_SCL_GPIO_Port    GPIOB
#define AHT20_SCL_Pin          GPIO_PIN_6
#define AHT20_SDA_GPIO_Port    GPIOB
#define AHT20_SDA_Pin          GPIO_PIN_7

#define AHT20_I2C_ADDR         0x70u

static void I2C_Delay(void)
{
    /*
     * 简单的软件延时，用于拉开 GPIO 翻转间隔。
     * 实际延时取决于主频、编译优化和函数调用开销。
     * 若时序不稳定，应使用更可靠的定时方式或逻辑分析仪检查。
     */
    for (volatile uint32_t i = 0; i < 20u; i++)
    {
        __NOP();
    }
}

static void I2C_SCL_High(void)
{
    HAL_GPIO_WritePin(AHT20_SCL_GPIO_Port,
                      AHT20_SCL_Pin,
                      GPIO_PIN_SET);
}

static void I2C_SCL_Low(void)
{
    HAL_GPIO_WritePin(AHT20_SCL_GPIO_Port,
                      AHT20_SCL_Pin,
                      GPIO_PIN_RESET);
}

static void I2C_SDA_High(void)
{
    HAL_GPIO_WritePin(AHT20_SDA_GPIO_Port,
                      AHT20_SDA_Pin,
                      GPIO_PIN_SET);
}

static void I2C_SDA_Low(void)
```

```
{
    HAL_GPIO_WritePin(AHT20_SDA_GPIO_Port,
                      AHT20_SDA_Pin,
                      GPIO_PIN_RESET);
}

static GPIO_PinState I2C_SDA_Read(void)
{
    return HAL_GPIO_ReadPin(AHT20_SDA_GPIO_Port, AHT20_SDA_Pin);
}

static void I2C_Start(void)
{
    I2C_SDA_High();
    I2C_SCL_High();
    I2C_Delay();

    /* SCL 为高时 SDA 从高变低，产生 START。 */
    I2C_SDA_Low();
    I2C_Delay();

    I2C_SCL_Low();
}

static void I2C_Stop(void)
{
    I2C_SDA_Low();
    I2C_Delay();

    I2C_SCL_High();
    I2C_Delay();

    /* SCL 为高时 SDA 从低变高，产生 STOP。 */
    I2C_SDA_High();
    I2C_Delay();
}

static uint8_t I2C_WriteByte(uint8_t data)
{
    for (uint8_t i = 0; i < 8u; i++)
    {
        I2C_SCL_Low();

        if ((data & 0x80u) != 0u)
        {
            I2C_SDA_High();
        }
        else
        {
```

```
            I2C_SDA_Low();
        }

        I2C_Delay();
        I2C_SCL_High();
        I2C_Delay();

        data <<= 1;
    }

    /* 释放 SDA，让从机有机会拉低线路返回 ACK。 */
    I2C_SCL_Low();
    I2C_SDA_High();
    I2C_Delay();

    I2C_SCL_High();
    I2C_Delay();

    uint8_t ack = (I2C_SDA_Read() == GPIO_PIN_RESET) ? 1u : 0u;

    I2C_SCL_Low();
    return ack;
}

static uint8_t I2C_ReadByte(uint8_t ack)
{
    uint8_t data = 0u;

    /* 主机释放 SDA，由从机驱动 SDA 输出数据。 */
    I2C_SDA_High();

    for (uint8_t i = 0; i < 8u; i++)
    {
        data <<= 1;

        I2C_SCL_High();
        I2C_Delay();

        if (I2C_SDA_Read() == GPIO_PIN_SET)
        {
            data |= 1u;
        }

        I2C_SCL_Low();
        I2C_Delay();
    }

    /*
     * ack 非零：主机发送 ACK，表示后面还要继续读。
```

```
     * ack 为零：主机发送 NACK，表示这是最后一个字节。
     */
    if (ack != 0u)
    {
        I2C_SDA_Low();
    }
    else
    {
        I2C_SDA_High();
    }

    I2C_SCL_High();
    I2C_Delay();
    I2C_SCL_Low();
    I2C_SDA_High();

    return data;
}

HAL_StatusTypeDef AHT20_Init(void)
{
    uint8_t status;

    I2C_SDA_High();
    I2C_SCL_High();
    HAL_Delay(40);

    /* AHT20 的 I2C 地址字节：7 位地址 0x38 左移一位后为 0x70。 */
    I2C_Start();

    if (I2C_WriteByte(AHT20_I2C_ADDR) == 0u)
    {
        I2C_Stop();
        return HAL_ERROR;
    }

    /* 读状态寄存器。 */
    if (I2C_WriteByte(0x71u) == 0u)
    {
        I2C_Stop();
        return HAL_ERROR;
    }

    status = I2C_ReadByte(0u);
    I2C_Stop();

    /*
     * 状态字节 bit[3] 为校准使能位。
     * 尚未校准时发送初始化命令 BE 08 00。
```

```
     */
    if ((status & 0x08u) == 0u)
    {
        I2C_Start();

        if (I2C_WriteByte(AHT20_I2C_ADDR) == 0u ||
            I2C_WriteByte(0xBEu) == 0u ||
            I2C_WriteByte(0x08u) == 0u ||
            I2C_WriteByte(0x00u) == 0u)
        {
            I2C_Stop();
            return HAL_ERROR;
        }

        I2C_Stop();
        HAL_Delay(10);
    }

    return HAL_OK;
}

HAL_StatusTypeDef AHT20_Read(float *temperature, float *humidity)
{
    uint8_t data[6];
    uint32_t raw_humidity;
    uint32_t raw_temperature;
    uint8_t status;

    if ((temperature == NULL) || (humidity == NULL))
    {
        return HAL_ERROR;
    }

    /* 发送触发测量命令 AC 33 00。 */
    I2C_Start();

    if (I2C_WriteByte(AHT20_I2C_ADDR) == 0u ||
        I2C_WriteByte(0xACu) == 0u ||
        I2C_WriteByte(0x33u) == 0u ||
        I2C_WriteByte(0x00u) == 0u)
    {
        I2C_Stop();
        return HAL_ERROR;
    }

    I2C_Stop();
    HAL_Delay(80);

    /* 重新寻址并读取状态与测量数据。 */
```

```
    I2C_Start();

    if (I2C_WriteByte(AHT20_I2C_ADDR | 0x01u) == 0u)
    {
        I2C_Stop();
        return HAL_ERROR;
    }

    status = I2C_ReadByte(1u);

    if ((status & 0x80u) != 0u)
    {
        /* busy 位仍为 1，测量尚未完成。 */
        (void)I2C_ReadByte(0u);
        I2C_Stop();
        return HAL_BUSY;
    }

    for (uint8_t i = 0; i < 5u; i++)
    {
        data[i] = I2C_ReadByte((i < 4u) ? 1u : 0u);
    }

    I2C_Stop();

    /*
     * 20 位湿度原始值：
     * data[1] 的低 4 位、data[2]、data[3]。
     */
    raw_humidity = ((uint32_t)data[1] << 12)
                 | ((uint32_t)data[2] << 4)
                 | ((uint32_t)data[3] >> 4);

    /*
     * 20 位温度原始值：
     * data[3] 的低 4 位、data[4]、data[5]。
     */
    raw_temperature = (((uint32_t)data[3] & 0x0Fu) << 16)
                    | ((uint32_t)data[4] << 8)
                    | (uint32_t)data[5];

    *humidity = ((float)raw_humidity * 100.0f) / 1048576.0f;
    *temperature = ((float)raw_temperature * 200.0f) / 1048576.0f
                 - 50.0f;

    return HAL_OK;
}
```

## 7. 调试过程与问题处理

最初串口显示 `T=-50.00 C, RH=0.00%`，说明读取到的原始值接近零，不能作为有效测量结果。随后增加了校准状态检查，未通过检查时输出 `AHT20 read failed`，避免把无效数据当成正常结果。

检查 GPIO 初始化后发现软件 $I^2C$ 总线空闲电平应为高电平，因此将 PB6/PB7 的初始输出设置为高，并配置为开漏输出和上拉。之后串口能够稳定输出约 24～30 ℃ 的温度和 55%～71%RH 的湿度数据。

CubeMX 重新生成代码时曾覆盖 `main.c` 顶部手动加入的头文件，导致 `OS_TCB`、`CPU_STK`、`OS_ERR` 和 AHT20 函数未定义。将头文件放回 `USER CODE BEGIN Includes` 区域后恢复编译。

另一个需要注意的现象是：`HAL_GPIO_TogglePin()` 每 3 秒改变一次电平，所以完整亮灭周期为 6 秒；如果要让完整亮灭周期为 3 秒，应把任务延时改为 1.5 秒。

## 8. 总结

次 STM32F103 移植 uC/OS-III 并驱动 AHT20 采集温湿度的实验，算是踩了不少坑，完整走完了 RTOS 移植、多任务开发、软件 I2C 传感器驱动的整套流程。

最开始我以为移植 uC/OS-III 就是把源码丢进工程就能跑，实际操作才发现细节特别多。不光要把内核、移植文件、配置文件分组添加到 Keil，头文件路径也要一个个配置好，稍微漏一个路径就会报一堆头文件找不到的错误。中断部分也很关键，需要修改`stm32f1xx_it.c`里的 PendSV 和 SysTick 中断函数，把中断入口对接给 uC/OS-III，如果这里写错，系统要么直接死机，要么任务完全无法调度。搞定底层移植之后，我创建了三个独立任务，分别实现 LED 闪烁、状态打印、传感器采集，依靠`OSTimeDlyHMSM`实现不同周期的并发运行，真正体会到 RTOS 多任务并行开发的便利，不用自己手写复杂的时间戳轮询逻辑。

实验过程里也遇到多任务共享串口的问题。一开始 Task2 和 Task3 都直接调用 HAL 库串口发送函数，串口输出的字符串经常乱掉、内容互相穿插。查阅资料后使用 uC/OS-III 的互斥量做资源保护，封装统一的串口发送函数，访问串口前申请互斥锁，发送结束再释放，就解决了多任务抢占带来的输出错乱问题。

AHT20 软件 I2C 驱动是本次实验最难调试的部分。一开始跑起来串口一直打印 T=-50.00℃、RH=0.00% 这种无效数据。排查了很久，先后检查接线、供电，才意识到软件 I2C 使用开漏输出模式，总线空闲时 SDA、SCL 必须维持高电平，硬件上也需要上拉电阻。同时 AHT20 上电后需要执行校准初始化，如果跳过校准，读出来的数据全部无效。把这些问题修正之后，才读到正常的温湿度数值。

做实验的过程中还踩了 CubeMX 的经典坑：重新生成工程代码的时候，放在非 USER CODE 区域的手写代码会被直接覆盖，好几次出现 uC/OS-III 相关类型、AHT20 函数未定义的编译报错。后

面写代码我就刻意把所有自己写的代码全部放在`USER CODE BEGIN / USER CODE END`保护块之间，避免代码被覆盖丢失。

做完整个实验，我对实时操作系统有了更具象的理解，不再只停留在课本概念。学会了任务创建、优先级设置、延时调度、互斥量资源保护；同时也掌握了软件 I2C 的时序逻辑，理解 I2C 起始、停止、应答信号的原理。也明白嵌入式开发不光是写代码，硬件接线、外设配置、时序、上拉这些硬件细节，都会直接决定程序能不能正常运行。后续如果继续优化，我可以尝试改用硬件 I2C 驱动传感器，减少 CPU 占用，也可以增加错误重连逻辑，传感器读取失败时自动重试，让程序鲁棒性变得更好。